

Universidad Técnica Estatal

Facultad de Ciencias de la Computación y Diseño Digital

Ingeniería en Software

Actividad Práctica de la Unidad

Comunicación entre Procesos, Relojes de Lamport, Tolerancia a Fallos,
Algoritmo Bully y Seguridad en un Sistema Distribuido con Java

Autor:

Cristhian Daniel Pacheco Cárdenas

Carrera:

Ingeniería en Software

Profesor:

Gleiston Ciceron Guerrero Ulloa

3 de junio de 2026

Índice

1. Introducción	2
2. Parte A — Comunicación entre procesos	2
2.1. Descripción	2
2.2. Servidor y cliente TCP	2
2.3. Framing de mensajes — <code>MessageUtil</code>	4
2.4. Protocolo gRPC — <code>nodo.proto</code>	5
2.5. Observaciones	5
3. Parte B — Relojes lógicos de Lamport	6
3.1. Descripción	6
3.2. La clase <code>LamportClock</code>	6
3.3. Simulación con tres procesos	7
3.4. Integración en el nodo gRPC	7
3.5. Observaciones	7
4. Parte C — Tolerancia a fallos	8
4.1. Descripción	8
4.2. La clase <code>MonitorHeartbeat</code>	8
4.3. Observaciones	9
5. Parte D — Coordinación: Algoritmo Bully	9
5.1. Descripción	9
5.2. La clase <code>BullyElection</code>	9
5.3. Observaciones	11
6. Parte E — Seguridad: Autenticación por Token	11
6.1. Descripción	11
6.2. La clase <code>AuthInterceptor</code>	11
6.3. Observaciones	12
7. Ejecución	13
8. Conclusiones	13
9. Repositorio en GitHub	13

1. Introducción

En esta práctica se construyó un sistema distribuido en Java que reúne cinco funcionalidades: comunicación por red, orden lógico de eventos, detección de fallos, elección automática de un coordinador y verificación de identidad de los clientes. Para la comunicación se usaron dos tecnologías: sockets TCP, que es la forma más directa de conectar programas por red, y gRPC, un sistema que permite llamar funciones de otro programa como si fueran propias.

El proyecto quedó organizado en tres paquetes:

- **comun**: clases reutilizables por todos los módulos (`LamportClock`, `Mensaje`, `MessageUtil`).
- **TCP**: servidor, cliente, monitor de latidos, simulación de Lamport y algoritmo Bully.
- **gRPC**: nodo con soporte gRPC, interceptor de autenticación y archivo `.proto`.

2. Parte A — Comunicación entre procesos

2.1. Descripción

Se implementaron dos canales de comunicación independientes: uno con sockets TCP planos y otro con gRPC. Un cliente puede conectarse, enviar un comando y recibir la respuesta del servidor.

2.2. Servidor y cliente TCP

El servidor escucha en el puerto 9000. Cada cliente que se conecta recibe su propio hilo de atención, de modo que varios clientes pueden estar conectados al mismo tiempo sin bloquearse entre sí.

```
ExecutorService pool = Executors.newFixedThreadPool(MAX_HILOS);

try (ServerSocket servidor = new ServerSocket(PUERTO)) {
    while (!Thread.currentThread().isInterrupted()) {
        Socket cliente = servidor.accept();
        int id = contadorClientes.incrementAndGet();
        pool.submit(new ManejadorCliente(cliente, id));
    }
}
```

El servidor reconoce tres comandos: **HORA** devuelve la hora actual, **ECO** repite el texto que se le envió, y **SALIR** cierra la conexión de ese cliente.

```
if (linea.equalsIgnoreCase("HORA")) {
    salida.println("HORA_ACTUAL:" + Servidor.timestamp());
} else if (linea.toUpperCase().startsWith("ECO")) {
    String mensaje = linea.length() > 4 ? linea.substring(4) : "(vacio)";
    salida.println("ECO[#" + idCliente + "]: " + mensaje);
} else if (linea.equalsIgnoreCase("SALIR")) {
    salida.println("ADIOS:hasta luego cliente #" + idCliente);
    break;
} else {
    salida.println("ERROR:comando desconocido -> " + linea);
}
```

El cliente se conecta, muestra un prompt y envía cada línea que el usuario escribe:

```
try (Socket socket = new Socket(HOST, PUERTO);
    BufferedReader entrada = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));
    PrintWriter salida = new PrintWriter(
        socket.getOutputStream(), true);
    Scanner teclado = new Scanner(System.in)) {

    while (true) {
        System.out.print("> ");
        String comando = teclado.nextLine().trim();
        salida.println(comando);
        System.out.println("Servidor: " + entrada.readLine());
        if (comando.equalsIgnoreCase("SALIR")) break;
    }
}
```

2.3. Framing de mensajes — MessageUtil

Para que dos mensajes enviados seguidos no se mezclen en el receptor, se diseñó MessageUtil. La idea es enviar primero cuatro bytes con la longitud del mensaje y luego los bytes del mensaje en sí, de modo que el receptor sepa exactamente cuántos bytes debe leer.

```
// Envío: longitud + datos
public static void send(Socket socket, String json) throws
    IOException {
    byte[] data = json.getBytes(StandardCharsets.UTF_8);
    DataOutputStream out = new DataOutputStream(socket.
        getOutputStream());
    out.writeInt(data.length);
    out.flush();
    // NOTA: falta escribir los bytes: out.write(data)
}

// Recepción: leer longitud, luego los datos exactos
public static String receive(Socket socket) throws IOException {
    DataInputStream in = new DataInputStream(socket.getInputStream());
    int length = in.readInt();
    byte[] data = new byte[length];
    in.readFully(data);
    return new String(data, StandardCharsets.UTF_8);
}
```

2.4. Protocolo gRPC — nodo.proto

En paralelo se definió un protocolo gRPC con cuatro operaciones: una para enviar comandos, otra para latidos, y dos para la elección de coordinador.

```
syntax = "proto3";
option java_package = "gRPC";
option java_multiple_files = true;

service NodoService {
  rpc Operar      (OperacionRequest)   returns (OperacionResponse);
  rpc Heartbeat   (HeartbeatRequest)   returns (HeartbeatResponse);
  rpc Eleccion    (EleccionRequest)    returns (EleccionResponse);
  rpc Coordinador (CoordinadorRequest) returns (CoordinadorResponse);
}

message OperacionRequest {
  string token      = 1;
  string payload    = 2;
  int64  lamportTime = 3;
  int32  nodoId     = 4;
}

message OperacionResponse {
  string resultado  = 1;
  int64  lamportTime = 2;
  int32  nodoId     = 3;
  bool   exito      = 4;
}

message HeartbeatRequest { int32 nodoId = 1; int64 lamportTime = 2; }
message HeartbeatResponse { int32 nodoId = 1; int64 lamportTime = 2; }
message EleccionRequest { int32 nodoIdEmisor = 1; int64 lamportTime = 2; }
message EleccionResponse { bool ok = 1; }
message CoordinadorRequest { int32 nodoIdCoordinador = 1; int64 lamportTime = 2; }
message CoordinadorResponse { bool recibido = 1; }
```

2.5. Observaciones

La comunicación TCP entre **Servidor** y **Cliente** funciona correctamente. Sin embargo, **MessageUtil** tiene un error: el método **send** escribe la longitud del mensaje pero nunca escribe los bytes del contenido, por lo que el receptor quedaría bloqueado esperando

datos que nunca llegan. Adicionalmente, esta clase nunca es llamada desde ningún otro archivo del proyecto. El canal TCP y el canal gRPC también quedaron como sistemas separados que no se integran entre sí.

3. Parte B — Relojes lógicos de Lamport

3.1. Descripción

Cada nodo debe llevar un contador lógico que siga tres reglas: incrementar ante cualquier evento propio, incrementar al enviar un mensaje adjuntando ese valor, y al recibir un mensaje fijar el reloj en el máximo entre el valor propio y el recibido, más uno.

3.2. La clase LamportClock

Esta clase implementa las tres reglas con un contador atómico, lo que la hace segura para usar con múltiples hilos al mismo tiempo.

```
// Regla 1: evento interno
public int eventoInterno(String descripcion) {
    int t = contador.incrementAndGet();
    System.out.printf("[%s] EVENTO INTERNO %-15s C=%d%n",
        nombreProceso, descripcion, t);
    return t;
}

// Regla 2: incrementar y adjuntar la marca al enviar
public int enviar(String destino, String mensaje) {
    int t = contador.incrementAndGet();
    System.out.printf("[%s] ENVIO -> %-8s msg='%s' C=%d%n",
        nombreProceso, destino, mensaje, t);
    return t;
}

// Regla 3: max(local, recibido) + 1 al recibir
public int recibir(String origen, int marcaMensaje, String mensaje)
{
    int nuevo;
    while (true) {
        int actual = contador.get();
        nuevo = Math.max(actual, marcaMensaje) + 1;
        if (contador.compareAndSet(actual, nuevo)) break;
    }
    System.out.printf("[%s] RECEPCION <- %-6s marcaMsg=%d C=%d%n",
```

```

        nombreProceso, origen, marcaMensaje, nuevo);
    return nuevo;
}

```

3.3. Simulación con tres procesos

Para demostrar el comportamiento, se construyó `SimulacionLamport` con tres procesos que se comunican por colas en memoria. Los mensajes van de P1 a P2, de P2 a P3, y de P3 de vuelta a P1.

```

// Proceso P1
LamportClock c = new LamportClock("P1");
c.eventoInterno("a");
int t = c.enviar("P2", "Hola P2");
p1aP2.offer(new int[]{t}); // envia la marca a P2
int[] marcaP3 = p3aP1.take(); // espera respuesta de P3
c.recibir("P3", marcaP3[0], "Respuesta de P3");
c.eventoInterno("f");

```

3.4. Integración en el nodo gRPC

El nodo gRPC también usa `LamportClock`. Cuando llega una operación, actualiza su reloj con la marca que viene en el mensaje y adjunta su propia marca en la respuesta.

```

public void operar(OperacionRequest req,
                  StreamObserver<OperacionResponse> obs) {

    long t = reloj.recibir("cliente", req.getLamportTime(), req.
getPayload());
    // ... procesamiento ...
    long tResp = reloj.eventoInterno("responder");

    obs.onNext(OperacionResponse.newBuilder()
                .setResultado(resultado)
                .setLamportTime(tResp)
                .build());
    obs.onCompleted();
}

```

3.5. Observaciones

El reloj de Lamport es correcto y funciona bien tanto en la simulación como en el nodo gRPC. El servidor TCP, en cambio, no lleva ningún reloj: responde texto plano sin

marca de tiempo lógica, por lo que la Parte B no queda cubierta en el canal TCP. Tampoco existe un log centralizado que combine eventos de todos los nodos en un orden total.

4. Parte C — Tolerancia a fallos

4.1. Descripción

Los nodos deben enviarse latidos periódicos entre sí. Si uno deja de responder, los demás deben detectarlo en un tiempo acotado y continuar operando con los nodos restantes.

4.2. La clase MonitorHeartbeat

El monitor verifica a intervalos regulares si un nodo responde intentando conectarse a su puerto TCP. Solo actúa cuando el estado cambia (de activo a caído o viceversa), llamando a una función de respuesta definida por quien lo usa.

```
private void verificarNodo() {
    boolean activo = false;
    try (Socket socket = new Socket()) {
        socket.connect(new InetSocketAddress(host, puerto),
            timeoutMs);
        activo = true;
        ultimoHeartbeat = Instant.now();
    } catch (Exception e) {
        log.warning("Heartbeat fallido para " + host + ":" + puerto
        );
    }

    boolean estadoAnterior = ultimoEstado.getAndSet(activo);
    if (activo != estadoAnterior) {
        String estado = activo ? "RECUPERADO" : "CAIDO";
        System.out.printf("[%s] Nodo %s:%d -> %s%n",
            LocalDateTime.now(), host, puerto, estado);
        callbackEstado.accept(activo);
    }
}
```

La verificación se repite automáticamente con un `ScheduledExecutorService`:

```

public void iniciar(ScheduledExecutorService scheduler) {
    tarea = scheduler.scheduleAtFixedRate(
        this::verificarNodo, 0, intervaloMs, TimeUnit.
        MILLISECONDS);
}

```

El handler gRPC para recibir latidos también actualiza el reloj de Lamport:

```

public void heartbeat(HeartbeatRequest req,
    StreamObserver<HeartbeatResponse> obs) {
    long t = reloj.recibir("Nodo-" + req.getNodoId(),
        req.getLamportTime(), "HB");
    obs.onNext(HeartbeatResponse.newBuilder()
        .setNodoId(nodoId).setLamportTime(t).build());
    obs.onCompleted();
}

```

4.3. Observaciones

El monitor solo verifica si el puerto TCP acepta conexiones; no llama al procedimiento gRPC `Heartbeat` definido en el proto, por lo que la marca de Lamport no se actualiza durante el monitoreo. Además, cuando detecta que un nodo cayó, solo muestra un mensaje: no dispara automáticamente una nueva elección Bully.

5. Parte D — Coordinación: Algoritmo Bully

5.1. Descripción

El algoritmo Bully elige como coordinador al nodo activo con el identificador más alto. Si el coordinador falla, el sistema debe iniciar una nueva elección de forma automática.

5.2. La clase `BullyElection`

Se simularon cinco procesos (P0 a P4) que se comunican por colas en memoria. Cada proceso recibe mensajes de tres tipos: `ELECTION` (alguien inicia una elección), `OK` (un proceso más grande responde que él tomará el control) y `COORDINATOR` (el ganador se anuncia a todos).

```

void procesarMensaje(MensajeBully msg) {
    switch (msg.tipo()) {
        case ELECTION -> {
            enviar(TipoMensaje.OK, msg.emisor());
            if (enEleccion.compareAndSet(false, true))
                iniciarEleccion();
        }
        case OK -> {
            enEleccion.set(false); // alguien mas grande existe, me retiro
        }
        case COORDINATOR -> {
            liderActual = msg.emisor();
            liderId.set(liderActual);
            enEleccion.set(false);
            System.out.printf("[P%d] El lider es: P%d%n", id, liderActual);
        }
    }
}

```

Cuando un proceso inicia una elección, envía ELECTION solo a los que tienen ID mayor y están activos. Si nadie responde en 1.5 segundos, se proclama líder y notifica a todos.

```

void iniciarEleccion() {
    boolean hayMasGrande = false;
    for (int i = id + 1; i < totalProcesos; i++) {
        if (procs[i] != null && procs[i].activo) {
            enviar(TipoMensaje.ELECTION, i);
            hayMasGrande = true;
        }
    }
    if (!hayMasGrande) {
        proclamarLider();
    } else {
        CompletableFuture.delayedExecutor(1500, TimeUnit.MILLISECONDS)
            .execute(() -> { if (enEleccion.get()) proclamarLider(); });
    }
}

```

La simulación hace que P4 arranque como líder, luego simula su fallo y P1 inicia una nueva elección; el resultado esperado es que P3 quede como nuevo coordinador.

```
procs[4].proclamarLider();  
Thread.sleep(1000);  
  
procs[4].simularFallo(); // cae el lider  
Thread.sleep(500);  
  
procs[1].enEleccion.set(true);  
procs[1].iniciarEleccion(); // nueva eleccion automatica  
  
Thread.sleep(3000);  
System.out.printf("El lider es: P%d%n", Proceso.liderId.get());
```

5.3. Observaciones

BullyElection tiene un error de compilación: la clase interna `Proceso` no fue declarada como `static`, y Java no permite instanciar una clase interna no estática desde un método estático como `main`. Sin esa corrección, el programa no compila. Además, el Bully de colas en memoria y los handlers gRPC de `NodoGrpc` son dos implementaciones separadas que no están conectadas entre sí.

6. Parte E — Seguridad: Autenticación por Token

6.1. Descripción

El cliente debe presentar un token antes de que el nodo acepte cualquier operación. Las solicitudes sin token válido deben rechazarse.

6.2. La clase AuthInterceptor

Se implementó un interceptor gRPC que revisa la cabecera `x-token` de cada mensaje antes de pasarlo al handler. Si el token no está o no es válido, responde con un error de autenticación y descarta la solicitud.

```

private static final Set<String> TOKENS_VALIDOS =
    Set.of("token-cliente-001", "token-cliente-002");

public <Q, R> ServerCall.Listener<Q> interceptCall(
    ServerCall<Q, R> call, Metadata headers,
    ServerCallHandler<Q, R> next) {

    String token = headers.get(TOKEN_KEY);

    if (token == null || !TOKENS_VALIDOS.contains(token)) {
        call.close(Status.UNAUTHENTICATED
            .withDescription("Token invalido o ausente"),
            headers);
        return new ServerCall.Listener<>() {};
    }

    return next.startCall(call, headers);
}

```

El interceptor se registra al construir el servidor, de modo que toda llamada gRPC pasa por él antes de llegar al servicio:

```

Server server = ServerBuilder.forPort(puerto)
    .intercept(new AuthInterceptor())
    .addService(new NodoGrpc(id))
    .build()
    .start();

```

6.3. Observaciones

El interceptor aplica a todas las llamadas gRPC sin distinción, incluyendo los latidos y los mensajes de elección entre nodos. Como esos mensajes no llevan token de cliente, serían rechazados con error de autenticación, impidiendo la comunicación interna del sistema. Por otro lado, el cliente y servidor TCP no implementan ningún mecanismo de autenticación: cualquier cliente TCP puede enviar comandos sin identificarse.

7. Ejecución

De todo el proyecto, los únicos componentes que compilan y se pueden ejecutar sin modificar ningún archivo son los siguientes:

- **SimulacionLamport**: se ejecuta solo en una terminal y termina en segundos de forma automática.
- **Servidor**: debe levantarse primero en su propia terminal y quedarse corriendo.
- **Cliente**: se ejecuta en una terminal adicional y requiere que el servidor ya esté corriendo.
- **MonitorHeartbeat**: se ejecuta en una terminal adicional con el servidor corriendo; dura 15 segundos y termina solo.

8. Conclusiones

La práctica permitió implementar los fundamentos de un sistema distribuido: comunicación por red, orden lógico de eventos, detección de fallos, elección de coordinador y autenticación. Se logró construir un servidor TCP funcional, una implementación correcta de los relojes de Lamport y un monitor de disponibilidad de nodos, además de definir un protocolo gRPC completo con todos los mensajes necesarios.

Sin embargo, el sistema quedó dividido en partes que no se comunican entre sí. El canal TCP y el gRPC, el Bully en memoria y la versión gRPC, y el monitor de latidos son módulos independientes. El aprendizaje más importante fue entender que en sistemas distribuidos no basta con que cada pieza funcione por separado: el verdadero desafío está en integrarlas de forma coordinada y tolerante a fallos.

9. Repositorio en GitHub

El programa de esta práctica práctica se encuentra disponible en:

<https://github.com/CristhianP03/Prueba-en-clase.git>

Capturas de pantalla

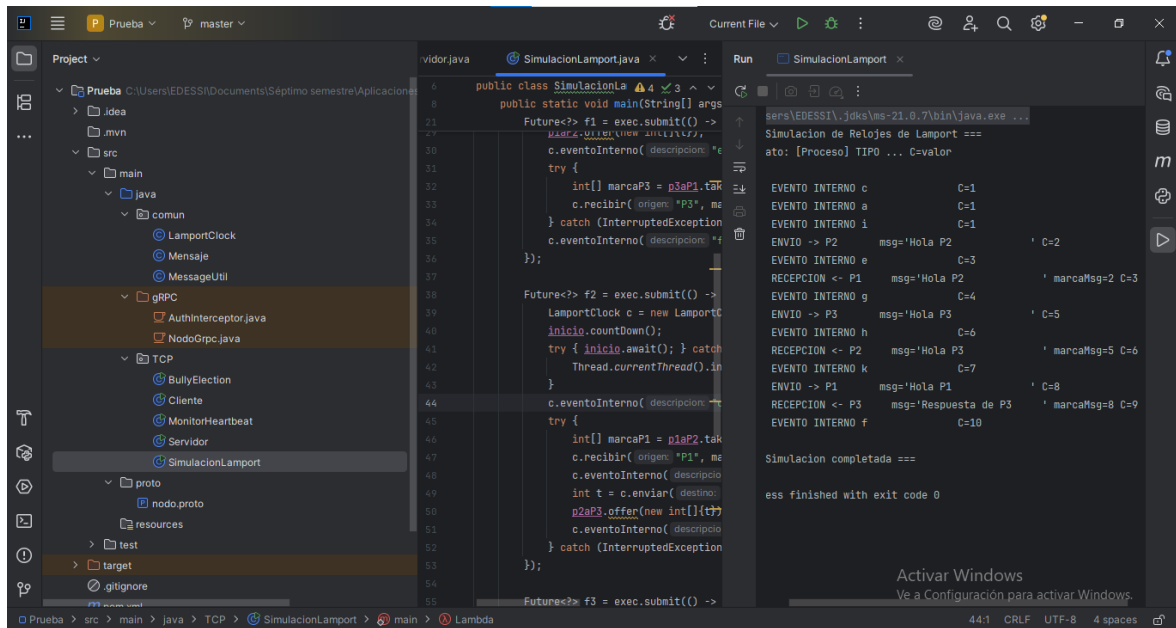


Figura 1: Simulación de relojes de Lamport — eventos de P1, P2 y P3

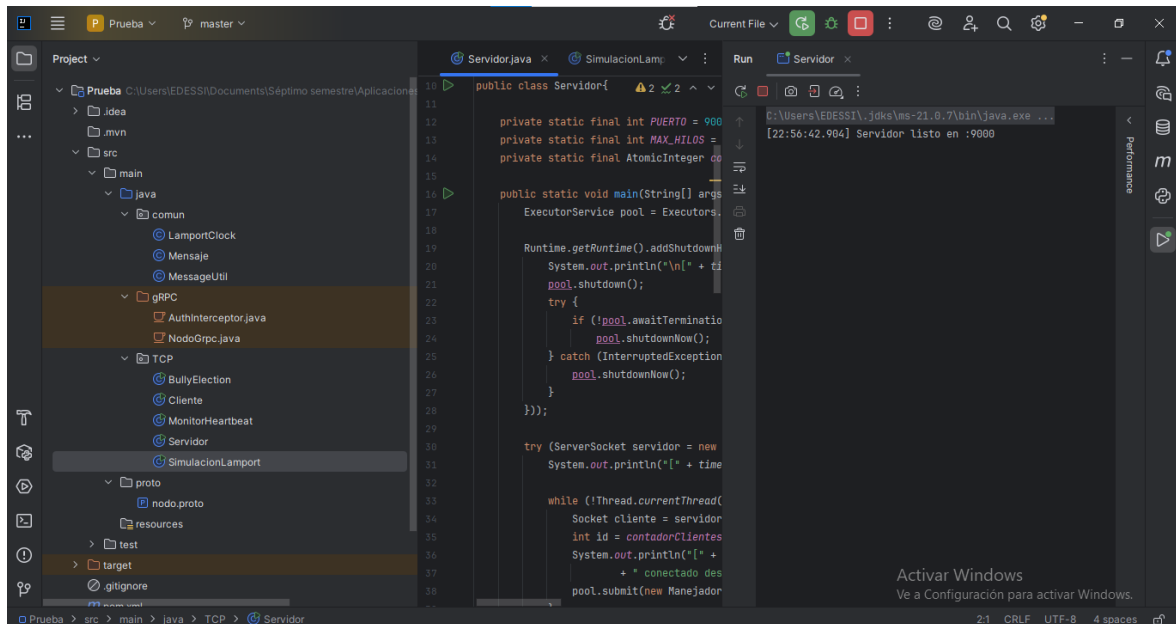


Figura 2: Servidor TCP iniciado en el puerto 9000

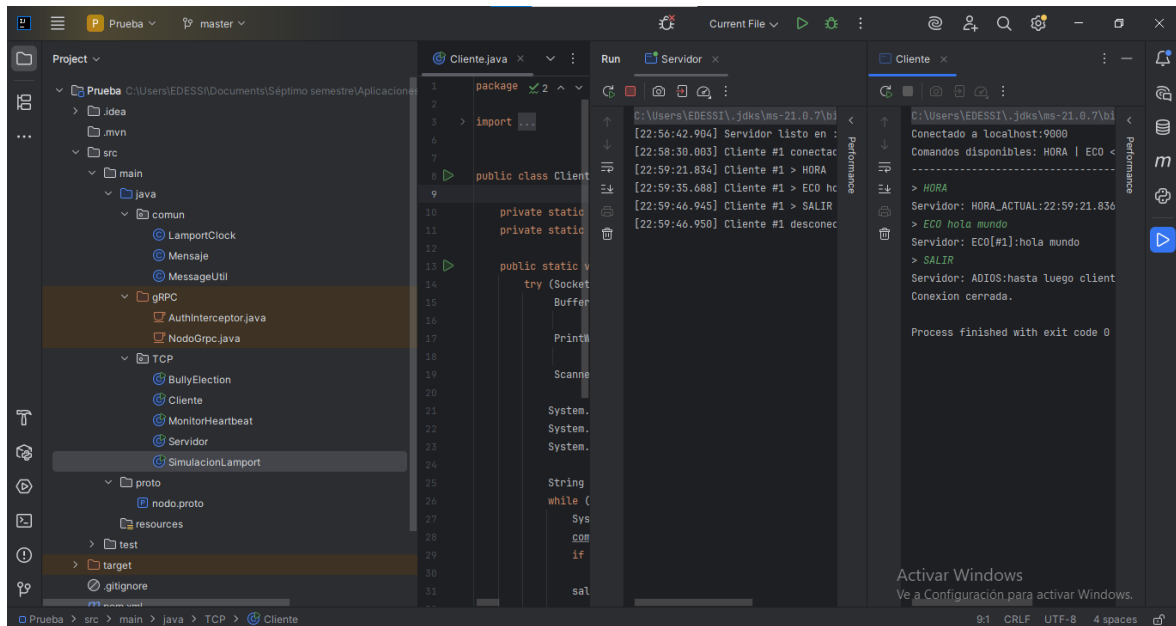


Figura 3: Interacción entre el cliente y el servidor TCP

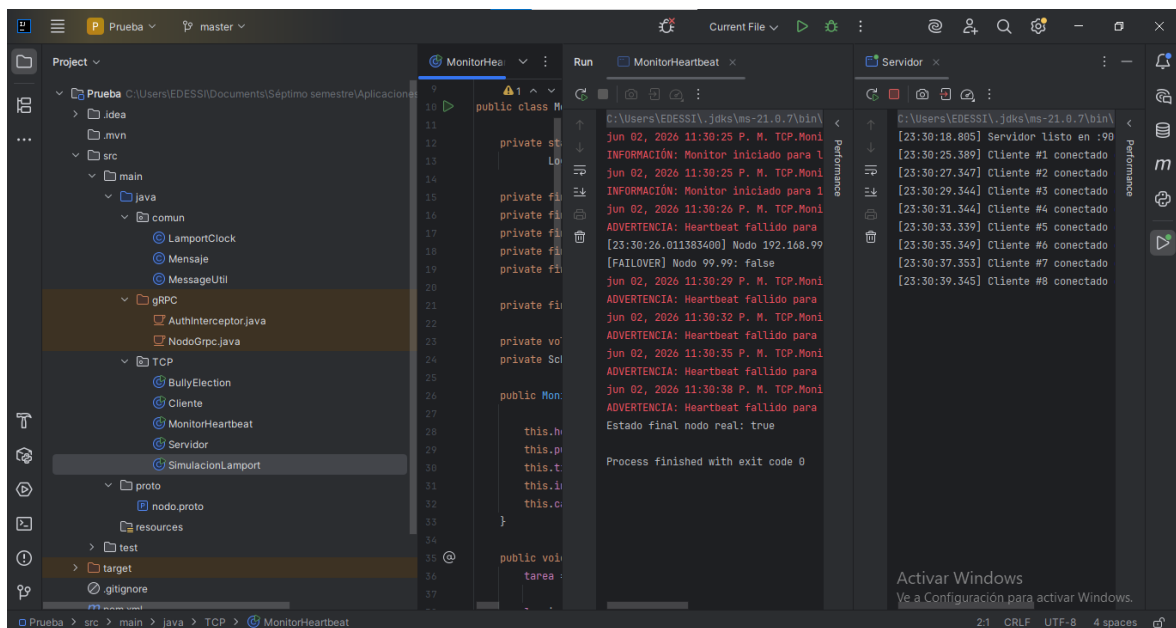


Figura 4: Monitor de Heartbeat reportando el estado del nodo

Rol de la Inteligencia Artificial

Durante la elaboración de este informe se utilizó una herramienta de inteligencia artificial como apoyo en la redacción. Su participación se limitó a correcciones menores en lo que respecta al texto y la estructura, como por ejemplo mejorar la redacción de algunos párrafos, reorganizar el orden de ciertas explicaciones y unificar el estilo del documento.



✓

Inicio (/) /

[← Atrás \(/alu_documentos?action=resumencuestionario&id=OOPPPQQRRRSSRTQOQL\)](#)



Revisión de intento

APLICACIONES DISTRIBUIDAS - SOFT-R - A - [7MO NIVEL] | CORTE 1

Estudiante	PACHECO CARDENAS CRISTHIAN DANIEL
Comenzado el	Martes, 2 de Junio de 2026, 07:57
Intento	1 / 1
Estado	Finalizado
Finalizado en	Martes, 2 de Junio de 2026, 08:07
Tiempo empleado	0:10:00
Calificación obtenida	5.58 / 20.00
Calificación final	2.79 / 10.00

PREGUNTA

1

Finalizado

1.00 / 1.00

Relaciones: Protocolo o algoritmo de coordinación y su propósito clave

Relacione la opción correcta:

Consenso Raft / Paxos

Logra Acuerdo Sobre Un Único Valor Mediante Quórum De Mayoría, Tolerando Caídas De Nodos. ✓

Commit en dos fases (2PC)

Garantiza Que Una Transacción Se Confirme En Todos Los Nodos O En Ninguno, Aunque Puede Quedar Bloqueado. ✓

Relojes de Lamport

Ordena Causalmente Los Eventos Sin Dependier De Un Reloj Físico Global. ✓

Algoritmo Bully

Elige Como Coordinador Al Nodo Activo De Mayor Identificador Tras Detectar Una Caída. ✓

La respuesta correcta es: Consenso Raft / Paxos → Logra acuerdo sobre un único valor mediante quórum de mayoría, tolerando caídas de nodos. , Relojes de Lamport → Ordena causalmente los eventos sin depender de un reloj físico global. , Commit en dos fases (2PC) → Garantiza que una transacción se confirme en todos los nodos o en ninguno, aunque puede quedar bloqueado si el coordinador cae. , Algoritmo Bully → Elige como coordinador al nodo activo de mayor identificador tras detectar una caída.

PREGUNTA

2

Finalizado

0.00 / 1.00

Dos centros de datos de un banco mantienen copias sincronizadas de las mismas cuentas. Una falla en el enlace de fibra deja a ambos centros sin comunicación entre sí, aunque cada uno sigue funcionando por separado y recibiendo clientes. Ante esta situación, el sistema decide bloquear todas las operaciones de retiro hasta que se restablezca el enlace, prefiriendo que ningún cliente pueda operar antes que arriesgarse a que la misma cuenta termine reflejando saldos distintos en cada centro. Según el teorema CAP, ¿qué par de propiedades prioriza esta decisión?

Seleccione una alternativa:

- ☒ a. Consistencia y disponibilidad (CA) ✕
- ☐ b. Consistencia y tolerancia a particiones (CP)
- ☐ c. Las tres propiedades simultáneamente
- ☐ d. Disponibilidad y tolerancia a particiones (AP).

La respuesta correcta es: Consistencia y tolerancia a particiones (CP)

PREGUNTA

3

Finalizado

0.00 / 1.00

En los relojes lógicos de Lamport se cumple que si el evento a ocurre antes que b ($a \rightarrow b$), entonces $C(a) < C(b)$. Sin embargo, observar que $C(x) < C(y)$ NO permite concluir que $x \rightarrow y$. ¿Por qué?

Seleccione una alternativa:

- ☐ a. Porque dos eventos concurrentes (sin relación causal) también pueden recibir marcas ordenadas, de modo que el orden total no implica causalidad.
- ☒ b. Porque $C(x) < C(y)$ siempre implica necesariamente $x \rightarrow y$, para la sincronización de los procesos. ✖
- ☐ c. Porque Lamport exige relojes físicos perfectamente sincronizados, es decir, que deben dar los mismos valores en los mismos tiempos.
- ☐ d. Porque los relojes de Lamport no son monótonos crecientes.

La respuesta correcta es: Porque dos eventos concurrentes (sin relación causal) también pueden recibir marcas ordenadas, de modo que el orden total no implica causalidad.

PREGUNTA

4

Finalizado

0.00 / 1.00

¿Cuál afirmación distingue correctamente un sistema distribuido de un sistema paralelo de memoria compartida?.

Seleccione una alternativa:

- ☐ a. En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes.
- ☐ b. El sistema distribuido no admite fallos parciales.
- ☐ c. El sistema distribuido siempre procesa más rápido.
- ☒ d. El sistema paralelo no puede tener múltiples procesadores. ✖

La respuesta correcta es: En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes.

PREGUNTA

5

Finalizado

0.00 / 1.00

Un servicio de distribución de archivos observa que, mientras más usuarios se conectan, más capacidad total (ancho de banda y almacenamiento) tiene disponible el sistema. ¿Qué modelo arquitectónico explica mejor esta propiedad?

Seleccione una alternativa:

- ☒ a. Cliente-servidor puro, porque el servidor central crece proporcionalmente con la demanda. ✖
- ☐ b. Modelo híbrido, porque depende siempre de un servidor central para todas las operaciones.
- ☐ c. Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.
- ☐ d. Cliente-servidor de tres capas, porque separa la lógica de negocio.

La respuesta correcta es: Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.

PREGUNTA

6

Finalizado

1.00 / 1.00

Un equipo necesita invocar métodos sobre objetos remotos entre dos máquinas virtuales de Java, pasando objetos como parámetros y manejando automáticamente la serialización. ¿Qué mecanismo es el más apropiado y por qué?

Seleccione una alternativa:

- ☐ a. TCP, porque garantiza la entrega ordenada y fiable de los objetos
- ☐ b. Llamada a procedimiento remoto clásica estilo C, porque soporta orientación a objetos de forma nativa.
- ☐ c. Sockets TCP crudos, porque ofrecen el mayor control de bajo nivel.
- ☒ d. Invocación remota de métodos, porque está diseñada para invocar métodos sobre objetos remotos en el ecosistema Java mediante stubs y serialización. ✓

La respuesta correcta es: Invocación remota de métodos, porque está diseñada para invocar métodos sobre objetos remotos en el ecosistema Java mediante stubs y serialización.

PREGUNTA

7

Finalizado

1.00 / 1.00

Un sistema replicado requiere un quórum de mayoría (más de la mitad de los nodos) para confirmar una operación y mantener la consistencia. Con 5 réplicas, ¿cuántos fallos por caída simultáneos puede tolerar sin perder la capacidad de formar quórum?

Seleccione una alternativa:

- ☐ a. 0, porque cualquier caída rompe el quórum.
- ☐ b. 4, porque basta con que una réplica siga viva.
- ☒ c. 2, porque el quórum de mayoría es 3 y se necesitan al menos 3 réplicas vivas ✓
- ☐ d. 3, porque con 2 réplicas vivas todavía hay mayoría.

La respuesta correcta es: 2, porque el quórum de mayoría es 3 y se necesitan al menos 3 réplicas vivas

PREGUNTA

8

Finalizado

0.00 / 1.00

Un usuario inicia sesión correctamente con su contraseña, pero el sistema le impide eliminar un archivo que pertenece a otro usuario. ¿Qué mecanismos de seguridad actúan en cada momento?

Seleccione una alternativa:

- ☐ a. Autenticación en ambos casos.
- ☒ b. Cifrado al iniciar sesión y autenticación al denegar la eliminación. ✖
- ☐ c. Autenticación al iniciar sesión y autorización al denegar la eliminación.
- ☐ d. Autorización en ambos casos.

La respuesta correcta es: Autenticación al iniciar sesión y autorización al denegar la eliminación.

PREGUNTA

9

Finalizado

1.00 / 1.00

Se compara un sistema centralizado con uno distribuido bajo la misma carga. ¿Cuál es una desventaja inherente al distribuido que el centralizado no presenta?

Seleccione una alternativa:

- ☒ a. La posibilidad de fallo parcial, donde unos nodos fallan y otros siguen operando, generando estados intermedios inconsistentes. ✔
- ☐ b. La existencia de un único punto de fallo total del sistema.
- ☐ c. La ausencia total de concurrencia.
- ☐ d. La imposibilidad de escalar horizontalmente.

La respuesta correcta es: La posibilidad de fallo parcial, donde unos nodos fallan y otros siguen operando, generando estados intermedios inconsistentes.

PREGUNTA

10

Finalizado

0.00 / 1.00

Un carrito de compras prioriza la disponibilidad (AP) y emplea consistencia eventual. Un usuario agrega un ítem y, al recargar de inmediato, no lo ve; segundos después sí aparece. ¿Cómo se explica esto correctamente?

Seleccione una alternativa:

- ☒ a. Es el comportamiento esperado: alta consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata devuelve datos actualizados. ✖
- ☐ b. Es el comportamiento esperado: bajo consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata puede devolver datos obsoletos.
- ☐ c. Es un fallo del sistema que viola el teorema CAP.
- ☐ d. El sistema cambió automáticamente a un modo CP e indica que el sistema no es tolerante a particiones

La respuesta correcta es: Es el comportamiento esperado: bajo consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata puede devolver datos obsoletos.

PREGUNTA

11

Finalizado

1.00 / 1.00

En el protocolo de 2PC, ¿cuál es su principal debilidad y por qué se produce?

Seleccione una alternativa:

- ☒ a. Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" pueden quedar bloqueados indefinidamente esperando la decisión final. ✓
- ☐ b. Que es desbloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" quedan libres del bloqueo indefinidamente esperando la decisión final.
- ☐ c. Que es no determinista en su resultado, en vista de que la comunicación entre las 2PC puede romperse.
- ☐ d. Que requiere relojes físicos sincronizados entre los participantes y que no garantiza la atomicidad de la transacción

La respuesta correcta es: Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" pueden quedar bloqueados indefinidamente esperando la decisión final.

PREGUNTA

12

Finalizado

0.00 / 1.00

Un sistema cifra todos los datos en tránsito con TLS, pero almacena las contraseñas en texto plano en la base de datos. Un atacante que compromete la base de datos obtiene todas las contraseñas. ¿Qué principio de seguridad se violó?

Seleccione una alternativa:

- ☐ a. La seguridad en reposo; el cifrado en tránsito no protege los datos almacenados.
- ☐ b. La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados.
- ☒ c. El no repudio o denegación de servicios (DOS), ya que el atacante cambió las contraseñas actuales. ✗
- ☐ d. La integridad de los datos en tránsito

La respuesta correcta es: La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados.

PREGUNTA

13

Finalizado

0.33 / 1.00

Sockets TCP

Relacione la opción correcta:

Sockets UDP

Datagramas Sin Garantía De Entrega Ni Orden, A Cambio De Menor Latencia.



Middleware orientado a mensajes (colas)

Invocación De Métodos Sobre Objetos Remotos Con Paso De Objetos Serializados Mediante Stubs.



Invocación remota de métodos (RMI)

Sincronización De Relojes Físicos De Los Nodos Mediante Señal Satelital.



La respuesta correcta es: Middleware orientado a mensajes (colas) → Invocación de métodos sobre objetos remotos con paso de objetos serializados mediante stubs. , Invocación remota de métodos (RMI) → Datagramas sin garantía de entrega ni orden, a cambio de menor latencia. , Sockets UDP → Flujo de bytes fiable y ordenado que no preserva los límites entre mensajes.

PREGUNTA

14

Finalizado

0.25 / 1.00

Relacione: Situación de seguridad y propiedad que se protege

Relacione la opción correcta:

Antes de conceder el acceso, el sistema valida que el usuario es quien dice ser mediante un segundo factor.

Confidencialidad



Un usuario ya autenticado intenta abrir un módulo administrativo y el sistema se lo deniega según su rol.

Autorización



Cada orden de pago se firma digitalmente para que el emisor no pueda negar después haberla emitido.

Integridad



El receptor recalcula un hash del mensaje y lo compara con el recibido para detectar si fue alterado en tránsito.

Autenticación



La respuesta correcta es: Un usuario ya autenticado intenta abrir un módulo administrativo y el sistema se lo deniega según su rol. → Autorización , Cada orden de pago se firma digitalmente para que el emisor no pueda negar después haberla emitido. → No repudio , El receptor recalcula un hash del mensaje y lo compara con el recibido para detectar si fue alterado en tránsito. → Integridad , Antes de conceder el acceso, el sistema valida que el usuario es quien dice ser mediante un segundo factor. → Autenticación

PREGUNTA

15

Sin contestar

0.00 / 1.00

Relacione el escenario y tipo de transparencia evidenciada:

Relacione la opción correcta:

Cien personas editan el mismo documento a la vez y el sistema preserva el orden correcto de los cambios sin que ellas lo perciban.

Elegir...



Un objeto se guarda y se recupera de una base de datos sin que el programador administre su almacenamiento en disco.

Elegir...



Una página sigue respondiendo con normalidad aunque un servidor del clúster se cayó, sin que el usuario lo note.

Elegir...



Un usuario abre un archivo sin enterarse de que físicamente reside en un servidor de otra ciudad.

Elegir...



La respuesta correcta es:

PREGUNTA

16

Sin contestar

0.00 / 1.00

Relaciones: Garantía o comportamiento observado y su modelo/concepto

Relacione la opción correcta:

Tras una escritura, algunas lecturas devuelven valores antiguos durante un lapso, pero todas terminan coincidiendo si cesan las escrituras.

Elegir...



Tras escribir un dato, cualquier lectura posterior en cualquier nodo devuelve ese valor de inmediato.

Elegir...

El sistema confirma una operación únicamente cuando más de la mitad de los nodos la han registrado.

Elegir...

Un proceso siempre ve sus propias escrituras en orden, aunque otros procesos las vean con retraso.

Elegir...

La respuesta correcta es:

PREGUNTA

17

Sin contestar

0.00 / 1.00

Un desarrollador usa sockets TCP para enviar por separado los mensajes "HOLA" y "MUNDO", pero en el receptor a veces llegan juntos como "HOLAMUNDO" en una sola lectura. ¿Cuál es la explicación correcta?

Seleccione una alternativa:

- ☐ a. TCP perdió y luego retransmitió un mensaje.
- ☐ b. TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente
- ☐ c. Es un error físico de la tarjeta de red
- ☐ d. El sistema cambió automáticamente a UDP.

La respuesta correcta es: TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente

PREGUNTA

18

Sin contestar

0.00 / 1.00

Una aplicación en Windows escrita en C# y otra en Linux escrita en Java deben intercambiar mensajes sin que ninguna conozca el sistema operativo ni el lenguaje de la otra. ¿Qué propiedad del middleware lo hace posible y cuál es su costo principal?

Seleccione una alternativa:

- ☐ a. Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización.
- ☐ b. Convierte el sistema distribuido en uno centralizado; el costo es la escalabilidad, dejando al sistema sin las posibilidades de crecer.
- ☐ c. Elimina por completo la latencia de red; el costo es el ancho de banda que se cree que es ilimitado por eso no se toma en cuenta su afectación.

- ☐ d. Garantiza que el sistema nunca fallará; el costo es el almacenamiento, que en nuestros días eso no es una limitación.

La respuesta correcta es: Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización.

PREGUNTA

19

Sin contestar

0.00 / 1.00

Un servicio se traslada de un servidor a otro durante la madrugada, cuando no hay clientes conectados; al reanudar la operación, los clientes lo siguen invocando con el mismo nombre lógico sin percibir cambio alguno. ¿Qué transparencia se evidencia principalmente?

Seleccione una alternativa:

- ☐ a. Transparencia de acceso.
- ☐ b. Transparencia de reubicación.
- ☐ c. Transparencia de migración.
- ☐ d. Transparencia de ubicación.

La respuesta correcta es: Transparencia de migración.

PREGUNTA

20

Sin contestar

0.00 / 1.00

Un arquitecto debe decidir entre consistencia fuerte (mediante consenso Raft) o consistencia eventual para un contador de "me gusta" en una red social con millones de operaciones por segundo. ¿Cuál es la decisión más razonable y por qué?

Seleccione una alternativa:

- ☐ a. Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después.
- ☐ b. Consistencia fuerte, porque el consenso Raft no introduce latencia adicional, siendo indiferente, porque ambos modelos rinden igual bajo carga elevada
- ☐ c. Consistencia general, porque el dominio tolera imprecisiones generales y conviene priorizar disponibilidad y rendimiento bajo niveles de carga promedio, siendo aceptable que el valor converja después.
- ☐ d. Consistencia fuerte, porque un conteo exacto en tiempo real es crítico para la seguridad del sistema.

La respuesta correcta es: Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después.

NAVEGACIÓN

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20												

Correcto ☐ Parcial ☐ Incorrecto ☐